📋 Resumo Executivo: Projeto AmpAI

> **ⓘ Visão Geral**
>
> O **AmpAI** é uma plataforma SaaS de engenharia elétrica de próxima geração. O produto foi projetado para eliminar a burocracia e obsolescência de softwares tradicionais (como ETAP e PTW), oferecendo uma interface web fluida inspirada no Obsidian, Excalidraw e AutoCAD, com automação baseada em Inteligência Artificial e rigor matemático alinhado às normas **IEC**.

🛠️ 1. Pilares da Stack Tecnológica

O ecossistema é dividido em três tecnologias especializadas que interagem de forma modular e isolada:

- **Core Administrativo e Regras de Negócio (.NET / C#):**
  - Gerencia a infraestrutura central do SaaS.
  - Controle de multi-inquilinato (Multi-tenancy), segurança/autenticação, bilhetagem/assinaturas (Stripe/Asaas).
  - Persistência e versionamento do Grafo JSON dos diagramas elétricos.
- **Motores de Cálculo Determinísticos (Rust):**
  - Executa as funções matemáticas pesadas e simulações elétricas.
  - Respostas na casa dos microssegundos com segurança de memória.
  - Garante imunidade a alucinações (resultado matemático puro e rígido).
- **Agente de Engenharia e RAG (Python):**
  - Microserviço em FastAPI utilizando frameworks como LangGraph ou LlamaIndex.
  - Processamento de linguagem natural e consultas semânticas ao banco de dados vetorial contendo as normas IEC indexadas.

🎨 2. Recursos do Workspace (Interface Gráfica)

A interface elimina modais complexos e foca na produtividade do usuário com três componentes integrados:

- **Grid do Unifilar:** Canvas interativo web (React Flow / HTML5 Canvas) de arrastar e soltar símbolos elétricos fixos e padronizados em formato SVG.
- **Painel de Propriedades Lateral:** Interface dinâmica inspirada no AutoCAD. Ao clicar em qualquer componente do unifilar, o painel atualiza instantaneamente as variáveis técnicas daquele item.
- **Editor de Memoriais Descritivos:** Ambiente flexível inspirado no Notion/Markdown para edição de relatórios e customização de templates com dados automáticos do projeto.

⏱️ 3. Estratégia de Implementação e Roadmap do Produto

O desenvolvimento e o modelo de monetização do SaaS serão divididos em 4 fases sequenciais para garantir um lançamento sustentável e validação contínua de mercado:

🔹 FASE 1: Core Administrativo e Gestão (.NET)

- [ ] Criar a estrutura base da API em ASP.NET Core.
- [ ] Modelar o Banco de Dados Relacional (SQL Server/PostgreSQL) focado em Multi-tenancy.
- [ ] Implementar autenticação corporativa e integração com gateway de pagamento (Stripe/Asaas).
- [ ] Desenvolver o frontend inicial do Canvas gerando o modelo JSON estável do grafo elétrico.

- **Modelo de Negócio:** Infraestrutura interna do sistema, controle de acessos de novos usuários.

🔹 FASE 2: SaaS Gratuito / Atração (Rust Básico)

- [ ] Desenvolver o microserviço stateless em Rust com framework Axum.
- [ ] Implementar funções de cálculos elétricos estáticos simples (potências, correntes, fator de potência, harmônicas).
- [ ] Conectar os endpoints do Rust à API do .NET para exibição em tempo real na interface.

- **Modelo de Negócio:** Camada pública gratuita para atração de tráfego orgânico de engenheiros.

🔹 FASE 3: SaaS por Assinatura / Comercial (Rust Avançado + Exportação)

- [ ] Implementar os motores avançados em Rust segundo as normas IEC (Curto-Circuito IEC 60909, Condutores IEC 60364, Seletividade).

- [ ] Criar pipeline de exportação de **PDF Técnico** de alta qualidade utilizando compilador Typst ou WeasyPrint.
- [ ] Desenvolver módulos de exportação da **Lista de Cargas** em Excel (.xlsx) e **Pranchas Unifilares** em CAD (.dxf/.dwg).

- **Modelo de Negócio:** Plano pago recorrente (SaaS Comercial B2B).

🔹 FASE 4: Premium por Token / Inteligência Artificial (Python + LLM)

- [ ] Configurar o microserviço Python FastAPI para gerenciamento dos agentes de IA.
- [ ] Criar a base de conhecimento RAG indexando as normas IEC em um banco vetorial (pgvector/Qdrant).
- [ ] Treinar/ajustar os prompts do Agente Engenheiro Eletricista para rodar auditorias em tempo real nos grafos JSON dos diagramas.

- **Modelo de Negócio:** Recursos Premium acionados via consumo de créditos/tokens (Copiloto de IA).

📌 Próximos Passos Recomendados

1. Definir o contrato do **Grafo JSON do Unifilar** (ele guiará a comunicação entre o .NET, Rust e Python).
2. Estruturar as tabelas do banco relacional com isolamento rígido por cliente (tenant_id).

#tags/ampai #arquitetura/saas #engenharia/eletrica #tecnologia/dotnet #tecnologia/rust #tecnologia/python